Язык программирования C++

Standard Template Library – STL
(Стандартная библиотека шаблонов)

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

STL

Библиотека стандартных шаблонов (STL) – набор согласованных обобщённых алгоритмов, контейнеров, средств доступа к их содержимому и различных вспомогательных функций в C++.

Внедрен в стандарт ISO / ANSI C++ в 1994г.

STL	
Контейнер	хранение набора объектов в памяти
Адаптер	адаптация контейнеров для обеспечения различного интерфейса
Итератор	обеспечение средств доступа к содержимому контейнера (обобщенный указатель)
Алгоритм	универсальные вычислительные подпрограммы, реализующие обобщенные алгоритмы
Функциональный объект	сокрытие функции в объекте для использования другими компонентами

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Контейнеры

Контейнер – это класс, предназначенный для хранения и организации данных какого-либо типа.

Примерами контейнеров являются массивы, векторы, списки.

Для каждого типа контейнера определены методы для работы с его элементами, не зависящие от конкретного типа данных.

Использование контейнеров позволяет значительно повысить надежность программ, их переносимость и универсальность, а также уменьшить сроки разработки.

При этом универсальность и безопасность использования контейнерных классов отражается на быстродействии программы: в зависимости от реализации компилятора снижение быстродействия может быть весьма значительным.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Последовательные контейнеры

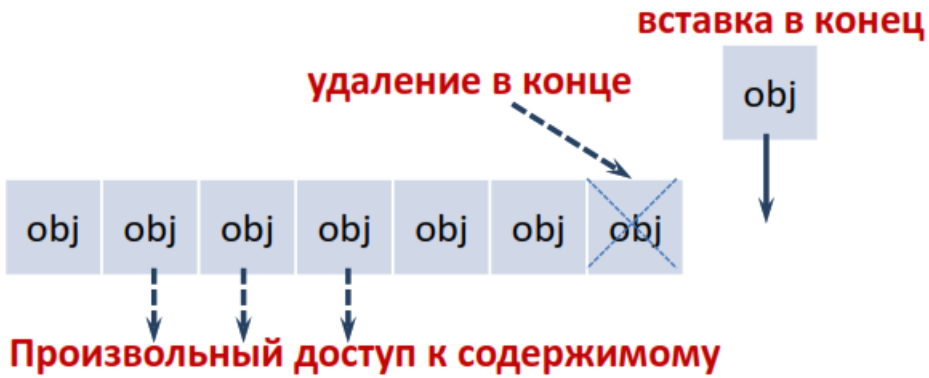
Последовательные контейнеры обеспечивают хранение конечного количества однотипных величин в линейной манере. Доступ к элементам осуществляется по индексам или последовательным перебором.

Контейнер	Описание
vector	С-подобный динамический массив произвольного доступа с автоматическим изменением размера при добавлении/удалении элемента. Элементы хранятся в непрерывной области памяти.
deque	Двусторонняя очередь похожа на vector, но с возможностью быстрой вставки и удаления элементов на обоих концах. С другой стороны, в отличие от vector, дэк не гарантирует расположение всех своих элементов в непрерывном участке памяти.
list	Двусвязный список, элементы которого хранятся в произвольных кусках памяти. Позволяет добавлять/удалять элементы за константное время.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Последовательные контейнеры

vector<T>



deque<T>

list<T>

никакого произвольного доступа – только последовательный

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Ассоциативные контейнеры

Ассоциативные контейнеры – это контейнеры, в которых объекты хранятся на основе некоторого ключа, который сопровождает каждый объект. Этот ключ используется для нахождения объекта в контейнере.

По умолчанию ассоциативные контейнеры выполняют сортировку элементов, используя оператор сравнения < .

Контейнер	Описание
set	Упорядоченное множество уникальных элементов. Обеспечивает стандартные операции над множествами (объединение, пересечение, вычитание).
multiset	То же что и set, но позволяет хранить повторяющиеся элементы.
map	Упорядоченный ассоциативный массив пар элементов, состоящих из ключей и соответствующих им значений. Ключи должны быть уникальны. Порядок следования элементов определяется ключами.
multimap	То же что и map, но позволяет хранить несколько одинаковых ключей.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Контейнеры-адаптеры

Адаптер контейнера – это шаблонный класс, служащий оболочкой для существующего контейнера STL при предоставлении других, обычно более ограниченных возможностей.

Контейнер	Основа	Описание
stack	deque	Стек – контейнер, в котором добавление и удаление элементов осуществляется с одного конца
queue	deque	Очередь – контейнер, с одного конца которого можно добавлять элементы, а с другого – вынимать
priority_queue	vector	Очередь с приоритетом, организованная так, что самый большой элемент всегда стоит на первом месте

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Итераторы

Итератор – это класс, объекты которого выполняют такую же роль по отношению к контейнеру, как указатели по отношению к массиву.

Указатель может использоваться в качестве средства доступа к элементам массива, а **итератор – в качестве средства доступа к элементам контейнера**.

Итераторы «склеивают» ядро STL в одну библиотеку.

Итераторы поддерживают абстрактную модель данных как **последовательности объектов**. Поэтому основное действие с итераторами – перебор элементов контейнера.

Понятия «нулевой итератор» не существует, а при организации циклов окончание фиксируется сравнением с концом последовательности.

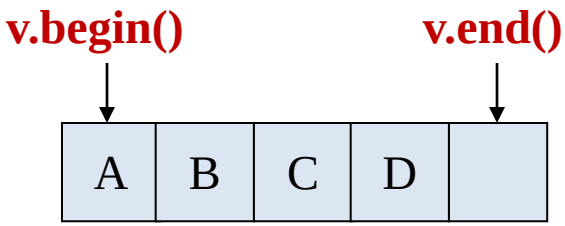
Каждый контейнер обеспечивает свои итераторы, поддерживающие стандартный набор итерационных операций со стандартными именами и смыслом.

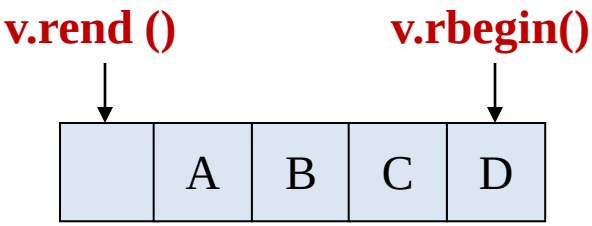
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Контейнеры и итераторы

Каждый контейнер имеет следующие методы для нахождения значений итераторов:

Метод	Описание
iterator **begin**();	возвращает итератор, который указывает на **первый** элемент последовательности
iterator **end**();	возвращает итератор, который указывает на элемент, **следующий за последним** элементом последовательности
reverse_iterator **rbegin**();	возвращает итератор, указывающий на **первый** элемент **в обратной** последовательности
reverse_iterator **rend**();	возвращает итератор, указывающий на элемент, **следующий за последним в обратной** последовательности





© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример вывода элементов string

```
#include <iostream>
#include <string>
using namespace std;

void main() {
  string str("Hello, world!");

  // string::iterator it;
  for (auto it = str.begin(); it != str.end(); ++it)
    cout << *it; // Hello, world!
  cout << endl;

  // string::reverse_iterator rIt;
  for (auto rIt = str.rbegin(); rIt != str.rend(); ++rIt)
    cout << *rIt; // !dlrow ,olleH
  cout << endl;
}
```

Вывод:

```
Hello, world!
!dlrow ,olleH
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Базовые операции над итераторами

Пусть **it1** и **it2** – объекты типа итератор.

К каждому итератору можно применить как минимум шесть ключевых операций:

- **`*it1`** – получение значения элемента, на который указывает итератор;
- **`++it1, it1++`** – переход к следующему элементу последовательности;
- **`it1 = it2`** – присваивание значения одного итератора другому;
- **`it1 == it2`** – проверка на равенство двух итераторов;
- **`it1 != it2`** – проверка на неравенство двух итераторов.

Пример:

```
some_container v;
some_container::iterator it1 = v.begin();
```

– такое присваивание верно независимо от типа контейнера v.

В новом стандарте можно писать auto it1 = v.begin();

Теперь *it1 – значение первого элемента контейнера v.

Замечание: При проходе последовательности как прямым, так и обратным итератором, переход к следующему элементу будет **`it1++`**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Категории итераторов

Не все виды итераторов поддерживают один и тот же набор операций.

Категория	Описание
Итераторы ввода и вывода (Input, Output)	Читают или пишут последовательность объектов. Однократное использование. Доступен только базовый набор операций.
Прямые (однонаправленные) итераторы (Forward)	Могут использоваться повторно для прохода по набору объектов в прямом направлении
Двунаправленные итераторы (Bidirectional)	Обеспечивают те же возможности, что и прямые итераторы, а также допускают операции **`--it`** и **`it--`** (можно проходить последовательность объектов в обратном направлении).
Итераторы произвольного доступа (RandomAccess)	Обладают теми же возможностями, что и двунаправленные итераторы, а также допускают следующие операции: **`it + n`** и **`it += n`**, **`it - n`** и **`it -= n`** **`it1 - it2`** **`it1 < it2`** и **`it1 <= it2`**, **`it1 > it2`** и **`it1 >= it2`** **`it[n]`** эквивалентен **`*(it + n)`**

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Алгоритмы

Алгоритмы STL – реализуют некоторые распространенные операции с контейнерами, которые не представлены методами каждого из контейнеров (например, сортировка, поиск, удаление элементов и т.д.).

Каждый алгоритм выражается шаблоном функции или набором шаблонов функций.

Операции, реализуемые алгоритмами, **являются универсальными** для любого из контейнеров и поэтому определены вне этих контейнеров.

Реализация алгоритмов **не использует имен** никаких конкретных **контейнеров**, а все действия над контейнером производятся через универсальные имена итераторов.

Зная, как устроены алгоритмы, можно писать свои собственные алгоритмы обработки, которые не будут зависеть от контейнера.

Все стандартные алгоритмы находятся **в пространстве имен std**, а их объявления – **в заголовочном файле < algorithm >**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Группы алгоритмов STL

Немодифицирующие – извлекают информацию из контейнера, но не модифицируют сам контейнер (ни элементы, ни порядок их расположения)	
find()	находит первое вхождение элемента с заданным значением
count()	количество вхождений элемента с заданным значением
for_each()	применяется некоторая операция к каждому элементу (не связано с изменением)
Модифицирующие – изменяют содержимое контейнера (изменяются сами элементы, либо их порядок, либо их количество)	
transform()	применяется некоторая операция к каждому элементу (каждый элемент изменяется)
reverse()	переставляет элементы в последовательности
copy()	создает новый контейнер – копию существующего
Сортировка	
sort()	простая сортировка
merge()	объединяет две отсортированные последовательности

https://ru.cppreference.com/w/cpp/algorithm

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Примеры алгоритмов STL

```
#include <algorithm>
#include <numeric> // для алгоритма accumulate

list<double> x;
list<string> ls;
vector<int> v(100);

// вычисление суммы через алгоритмы
sum = accumulate(x.begin(), x.end(), 0.0);

// поиск значения "К8" в диапазоне от ls.begin() до ls.end()
list<string>::const_iterator it;
it = find(ls.begin(), ls.end(), "К8");

// заполнение всех элементов вектора v значением 0
fill(v.begin(), v.end(), 0);

// замена всех элементов вектора v равных -1 на 1
replace(v.begin(), v.end(), -1, 1);
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Состав контейнеров

В каждом классе-контейнере определен набор методов для работы с данными, причем все контейнеры поддерживают **стандартный набор базовых операций**.

Базовая операция контейнера (базовый метод) – метод класса, имеющий во всех контейнерах одинаковое имя, одинаковый прототип и семантику (их примерно 15–20).

Например:

- метод **push_back()** помещает элемент в конец контейнера;
- метод **size()** выдает текущее количество элементов в контейнере.

Базовыми методами можно пользоваться одинаково независимо от того, в каком конкретно контейнере находятся элементы. Можно также менять контейнеры, не меняя тела функций, работающих с ними.

Операции, которые не могут быть эффективно реализованы для всех контейнеров, не включаются в набор общих операций.

Например, обращение по индексу введено для контейнера **vector**, но не для **list**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Некоторые общие методы

Операция	Метод	vector	deque	list
Вставка в начало	push_front	–	+	+
Удаление из начала	pop_front	–	+	+
Вставка в конец	push_back	+	+	+
Удаление из конца	pop_back	+	+	+
Вставка в произвольное место	insert	(+)	(+)	+
Удаление из произвольного места	erase	(+)	(+)	+
Произвольный доступ к элементу	[], at	+	+	–

Знак – означает, что соответствующая операция не реализована.
Знак + означает, что соответствующая операция реализуется за постоянное время, не зависящее от количества n элементов в контейнере.
Знак (+) означает, что соответствующая операция реализуется за время, пропорциональное n.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Типы, используемые в контейнерах

Каждый контейнер в своей **public** части содержит серию определений через **typedef**, где введены стандартные имена типов.

typedef позволяет создавать псевдонимы для типов данных. Например:

```
typedef unsigned int size_t; // size_t – псевдоним
```

value_type	тип элемента
size_type	тип, используемый для индексации
iterator, const_iterator	итератор
reverse_iterator, const_reverse_iterator	обратный итератор
pointer, const_pointer	указатель на элемент
reference, const_reference	ссылка на элемент

Эти имена определяются внутри каждого контейнера, скрывая реальные типы. Это позволяет составить код, который будет работать с любым контейнером.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс array

std::array – это фиксированный массив, который не распадается в указатель при передаче в функцию. std::array определяется в заголовочном файле <array>. Представлен в C++11.

Объявление:

```
array<int, 4> marr; // объявляем массив типа int длиной 4
```

Подобно обычным статическим массивам, длина array должна быть установлена во время компиляции. array можно инициализировать с использованием списка инициализаторов или uniform-инициализации:

```
array<int, 4> marr1 = { 8,6,4,1 }; // список инициализаторов
array<int, 4> marr2{ 8,6,4,1 }; // uniform-инициализация
```

В отличие от стандартных статических массивов, в array вы не можете пропустить (не указывать) длину массива.

Также можно присваивать значения массиву с помощью списка инициализаторов:

```
myarray = { 0,1,2,3 }; // ок
myarray = { 8,6 };     // ок, элементам 2 и 3 присвоен нуль!
myarray = { 0,1,3,5,7,9 }; // нельзя, слишком много
                           // элементов в списке инициализаторов!
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс array. Размер и сортировка

С помощью метода size() можно **узнать длину массива**:

```
array<double, 4> myarray = { 8.0, 6.4, 4.3, 1.9 };
cout << "длина: " << myarray.size();
```

Поскольку std::array не распадается в указатель при передаче в функцию, то метод size() будет работать, даже если его вызвать из другой функции:

```
void printLength(const array<double, 4> &myarray)
{
  cout << "длина: " << myarray.size();
}
```

! Всегда передавайте std::array в функции по обычной или по константной ссылке.

С помощью метода sort() можно **отсортировать массив**:

```
// сортировка массива по возрастанию
sort(myarray.begin(), myarray.end());
// сортировка массива по убыванию
sort(myarray.rbegin(), myarray.rend());
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Создание

Создание **пустого** контейнера:

```
vector<int> mydata; // Контейнер с нулевой ёмкостью
mydata.push_back(99); // добавляет новый элемент в конец
```

Создание контейнера **заданного размера:**

```
vector<int> mydata(100); /* вектор, содержащий 100
                 элементов, инициализированных значением 0 */
vector<int> mydata(100, -1) /* все элементы вектора будут
                                  содержать значение -1 */
```

Создание пустого контейнера **с зарезервированной памятью:**

```
vector<int> mydata;
mydata.reserve(100); /* выделение достаточной памяти для
                              размещения 100 элементов */
```

Создание контейнера **на основе внешнего массива:**

```
double data[] = {1.5, 2.5, 3.5, 4.5, 5.5., 6.5, 7.5, 8.5};
vector<double> mydata(data, data+8); /* вектор из элементов
                                        data[0]… data[7] */
```

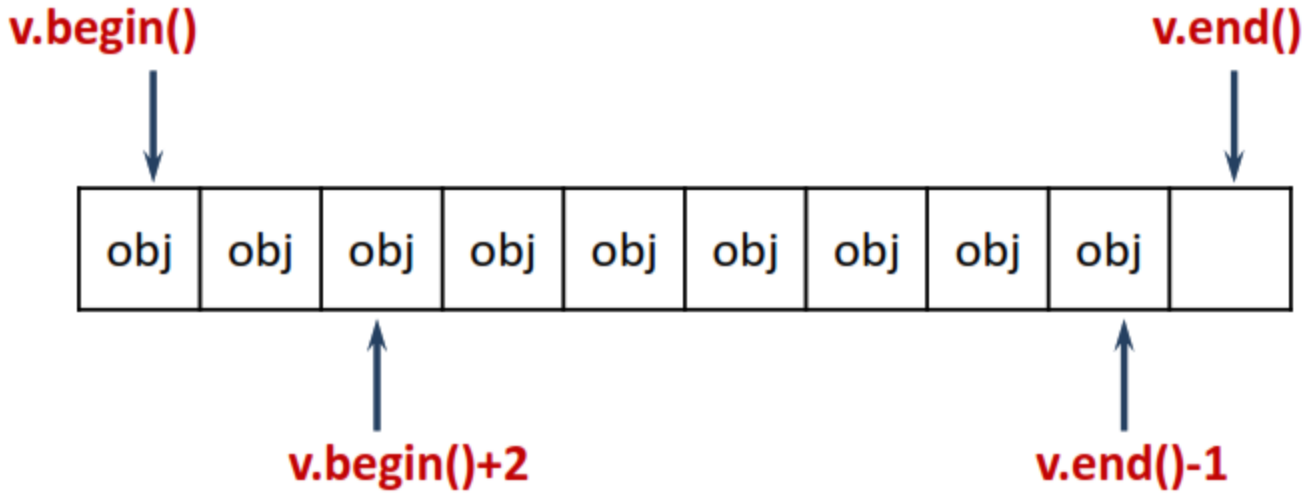

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Создание

Создание контейнера **на основе другого контейнера:**

```
// вектор values будет дублировать вектор mydata:
vector<double> values(mydata.begin(), mydata.end());

// вектор инициализируется элементами с третьего по седьмой
// из вектора mydata:
vector<double> values(mydata.begin() + 2, mydata.end() – 1);
```



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Ёмкость/размер

В векторе следует различать понятия ёмкость и размер:

Ёмкость – это максимальное количество элементов, под которое уже зарезервирована динамическая память. Узнать можно так: **v.capacity();**

Размер – это количество элементов, которые в данный момент хранятся в векторе. Узнать можно так: **v.size();**

При добавлении новых элементов не произойдёт выделение памяти, если **Ёмкость ≥ Размер + Количество_новых_элементов**. В противном случае, в новом месте будет выделена память с запасом. Туда скопируются все элементы, включая новые. Старая память будет освобождена.

Проверка **пустоты** вектора**:**

```
data.size() == 0
bool e = data.empty()
```

Изменение размера существующего вектора:

```
data.resize(N, K=0); // N – новое количество элементов,
                     // K – значение для новых элементов
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Обращение к элементам

По индексу:

```
vector<int> numbers(10);
for (vector<int>::size_type i = 0; i < numbers.size(); i++)
    cout << " " << numbers[i];
```

Метод at():

```
for (vector<int>::size_type i = 0; i < numbers.size(); i++)
    cout << " " << numbers.at(i);
```

Метод at() отличается наличием проверки индекса. Если указать номер аргумента, выходящий за пределы допустимого диапазона, то будет сгенерировано исключение out_of_range.

Обращение к **первому или последнему** элементу

```
cout << "Значение первого элемента: " << numbers.front();
cout << "Значение последнего: " << numbers.back();
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Вставка элементов

```
vector<int> vec(4, 99); // содержимое вектора: 99 99 99 99
```

Добавление элемента в конец вектора методом **push_back()**:

```
vec.push_back(100); // содержимое вектора: 99 99 99 99 100
```

Вставка новых элементов в любое место вектора методом **insert()**:

Вариант 1. Вставка **единственного** элемента:

```
vec.insert(vec.begin() + 1, 88);
// содержимое контейнера: 99 88 99 99 99 100
```

Вариант 2. Вставка **нескольких идентичных** элементов:

```
vec.insert(vec.begin() + 2, 3, 77);
// содержимое контейнера: 99 88 77 77 77 99 99 99 100
```

Вариант 3. Вставка **из другого контейнера** элементов:

```
vector<int> V(5, 22); // содержимое: 22 22 22 22 22
V.insert(V.begin() + 1, vec.begin() + 1, vec.begin() + 5);
// содержимое: 22 88 77 77 77 22 22 22 22
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Удаление элементов

Исходный вектор:

```
int data[] = { 1, 2, 3, 4, 5, 6, 7 };
vector<int> vec(data, data + 7);
// содержимое контейнера: 1 2 3 4 5 6 7
```

Удаление **последнего** элемента методом **pop_back()** (Размер сокращается на 1, ёмкость остается прежней):

```
vec.pop_back(); // содержимое контейнера: 1 2 3 4 5 6
```

Удаление элементов **из любой позиции** методом **erase()**:

Вариант 1. Удаление **единственного** элемента в заданной позиции:

```
vec.erase(vec.end() - 2);
// содержимое контейнера: 1 2 3 4 6
```

Вариант 2. Удаление **нескольких** элементов:

```
vec.erase(vec.begin() + 1, vec.begin() + 3);
// содержимое контейнера: 1 4 6
```

Удаление **всех элементов** методом **clear()** (Размер становится нулевым, ёмкость не меняется):

```
vec.clear()
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Замена элементов

Обмен содержимого двух векторов методом **swap()**:

```
vector<int> first(5, 77); // содержит 77 77 77 77 77
vector<int> second(7, -1); // содержит -1 -1 -1 -1 -1 -1 -1
first.swap(second) или second.swap(first);
```

Замена содержимого вектора **другой последовательностью** – **assign()**:

Вариант 1. Замена вектора **вектором**:

```
vector<double> values;
for (int i = 1; i <= 5; i++) values.push_back(2.5*i);
// 2.5, 5.0, 7.5, 10.0, 12.5
vector<double> newdata(3, 3.5); // 3.5, 3.5, 3.5
newdata.assign(values.begin() + 1, values.end() - 1);
```

Вариант 2. Замена вектора **элементами массива**:

```
double mydata[] = { 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0 };
newdata.assign(mydata, mydata + 3); // 1.0, 2.0, 3.0
```

Вариант 3. Замена вектора **набором одинаковых элементов**:

```
newdata.assign(5, 99.5); // 99.5 99.5 99.5 99.5 99.5
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Сохранение объектов

Минимальная спецификация для класса T, обеспечивающая совместимость с вектором или любым последовательным контейнером:

```
class T
{
public:
  T();                          // стандартный конструктор
  T(const T & t);               // конструктор копирования
  ~T();                         // деструктор
  T& operator = (const T&t); // оператор присваивания
};
```

Создание вектора с пользовательским классом:

```
vector<T> V;
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Хранение указателей

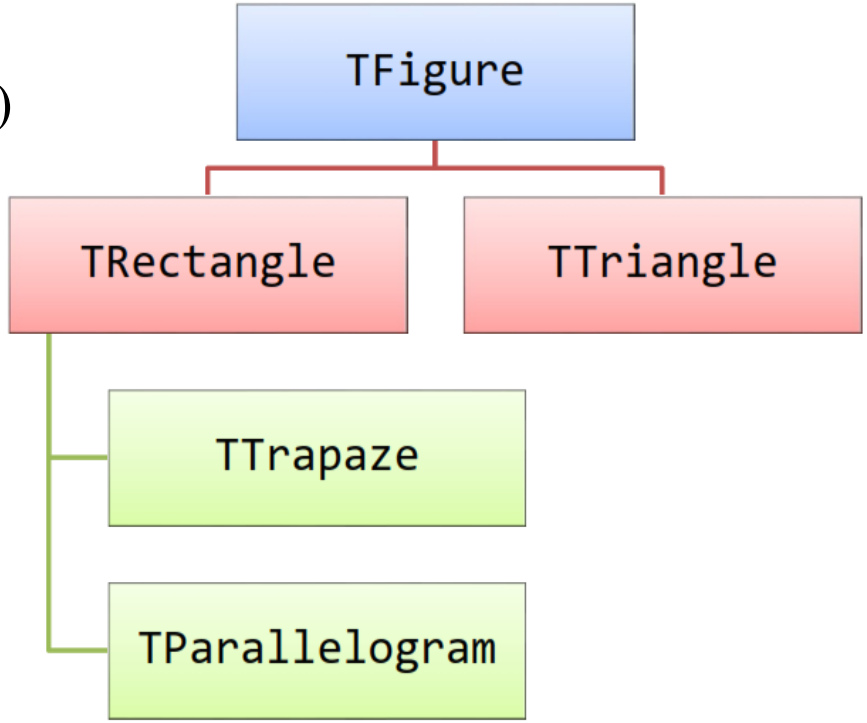
Причины хранения в векторе указателей на объекты:

1. Если объекты велики, то копирование каждого объекта при его добавлении в контейнер может повлечь за собой существенные накладные расходы.
2. Использование абстрактного класса.

Пример.

Абстрактный класс TFigure (фигура) и его производные классы:

- TTriangle (треугольник)
- TRectangle (прямоугольник)
- TTrapaze (трапеция)
- TParallelogram (параллелограмм)




```
vector<TFigure*> figures(2);
figures[0] = new TTrapaze();
figures[1] = new TTriangle();
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс vector. Сортировка указателей

```
bool compare(TFigure *F1, TFigure *F2) {
  return (F1->getArea() < F2->getArea());
}

void main() {
  vector<TFigure*> figures;
  ...
    case 'T':
      cin >> a >> b >> c;
      figures.push_back(new TTriangle(a, b, c));
      break;
    case 'R':
      cin >> a >> b;
      figures.push_back(new TRectangle(a, b));
      break;
  ...
  sort(figures.begin(), figures.end(), compare);
}
```

Компаратор

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс list. Создание

Двусвязный список, элементы которого хранятся в произвольных кусках памяти. Для его использования необходимо подключить заголовочный файл <list>.

Объявление:

```
list<тип> имя;
```

Создание списка:

```
list<int> list1;    // пустой список
list<int> list2(5); /* список list2 состоит из 5 чисел,
                       каждый элемент имеет значение по умолчанию */
list<int> list3(5, 2);  /* список list3 состоит из 5 чисел,
                           каждое число равно 2 */
list<int> list4{ 1, 2, 4, 5 };  /* список list4 состоит из
                                   чисел 1, 2, 4, 5 */
list<int> list5 = { 1, 2, 3, 5 }; /* список list5 состоит из
                                     чисел 1, 2, 4, 5 */
list<int> list6(list4);   // список list6 - копия list4
list<int> list7 = list4;  // список list7 – копия list4
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс list. Добавление элементов

Для добавления элементов в контейнер list применяется ряд методов:

- **push_back(val)**: добавляет значение val в конец списка.
- **push_front(val)**: добавляет значение val в начало списка.
- **insert(pos, val)**: вставляет элемент val на позицию, на которую указывает итератор pos. Возвращает итератор на добавленный элемент.
- **insert(pos, n, val)**: вставляет n элементов val начиная с позиции, на которую указывает итератор pos. Возвращает итератор на первый добавленный элемент. Если n = 0, то возвращается итератор pos.
- **insert(pos, begin, end)**: вставляет начиная с позиции, на которую указывает итератор pos, элементы из другого контейнера из диапазона между итераторами begin и end. Возвращает итератор на первый добавленный элемент.

В C++11 добавились методы, создающие новые объекты на месте:

- **emplace_back(val)**: создаёт объект из значения val в конце списка.
- **emplace_front(val)**: создаёт объект из значения val в начале списка.
- **emplace(pos, val)**: создаёт объект из значения val на позиции, на которую указывает итератор pos.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс list. Получение элементов

Для list **не определена** операция обращения по индексу **[]** или метод **at()**.

Для контейнера list можно использовать **методы front() и back()**, которые возвращают соответственно **первый и последний элементы**.

Чтобы обратиться к элементам, которые находятся в середине (после первого и до последнего элементов), придется выполнять **перебор элементов с помощью циклов** или итераторов:

```
std::list<int> numbers = { 1, 2, 3, 4, 5 };
int first = numbers.front();  // 1
int last = numbers.back();  // 5

// перебор в цикле
for (int n : numbers)
    std::cout << n << "\t";

// перебор с помощью итераторов
for (auto it = numbers.begin(); it != numbers.end(); it++)
{
    std::cout << *it << "\t";
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс list. Удаление элементов

Для удаления элементов в контейнере list применяется ряд методов:

- **clear()**: удаляет все элементы.
- **pop_back()**: удаляет последний элемент.
- **pop_front()**: удаляет первый элемент.
- **erase(pos)**: удаляет элемент, на который указывает итератор pos. Возвращает итератор на элемент, следующий после удаленного.
- **erase(begin, end)**: удаляет элементы из диапазона, на начало и конец которого указывают итераторы begin и end. Возвращает итератор на элемент, следующий после последнего удаленного.

```
std::list<int> numbers = { 1, 2, 3, 4, 5 };
numbers.pop_front();    // numbers = { 2, 3, 4, 5 }
numbers.pop_back();     // numbers = { 2, 3, 4 }
numbers.clear();        // numbers = {}
numbers = { 1, 2, 3, 4, 5 };
auto iter = numbers.begin(); // указатель на первый элемент
numbers.erase(iter);    // удаляем первый элемент
                        // numbers = { 2, 3, 4, 5 }
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс list. Размер

Для получения размера списка можно использовать метод **size()**:

```
std::list<int> numbers = { 1, 2, 3, 4, 5, 6 };
int size = numbers.size();   // 6
```

Метод **empty()** позволяет узнать, пуст ли список:

```
if (numbers.empty())
  cout << "Этот список пуст" << endl;
else
  cout << "Этот список содержит элементы" << endl;
```

С помощью функции **resize()** можно изменить размер списка:

resize(n): оставляет в списке n первых элементов. Если список содержит больше элементов, то он усекается до первых n элементов. Если размер списка меньше n, то добавляются недостающие элементы и инициализируются значением по умолчанию.

resize(n, value): аналогично предыдущему, но если размер списка меньше n, то недостающие элементы будут со значением value.

```
numbers.resize(4);      /* оставляем первые четыре элемента -
                           numbers = {1, 2, 3, 4} */
numbers.resize(6, 8);  // numbers = {1, 2, 3, 4, 8, 8}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Структура pair

Для хранения пары «ключ – элемент» в ассоциативном контейнере используется шаблон pair, позволяющий хранить два разнородных объекта как единое целое. Pair описан в заголовочном файле <utility>:

```
template <class T1, class T2>
struct pair {
  typedef T1 first_type; // псевдоним для типа первого поля
  typedef T2 second_type; // псевдоним для типа второго поля
  T1 first; // первое поле (используется как ключ)
  T2 second; // второе поле (элемент)
  // перегрузки операций отношения ==, !=, <, <=, >, >=
};
```

```
pair<int, string> count_word(15, "Ivanov");
```

Доступ к элементам пары:

```
if (count_word.first > 10 && count_word.second == "Ivanov")
    cout << "Ivanov встречается больше 10 раз" << endl;
count_word.first = 20;
count_word.second = "Romanov";
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Функция std::make_pair

При использовании std::pair вам необходимо указать типы обоих элементов, тогда как std::make_pair создаст пару с типом передаваемых ей элементов.

```
// std::pair<string, string>
std::pair<string, string> author("Ivan", "Ivanov");
// std::pair<const char*, const char*>
auto author = make_pair("Ivan", "Ivanov");

// std::pair<int, string>
std::pair<int, string> count_word(15, "Ivan");
// std::pair<int, const char*>
auto count_word = make_pair(15, "Ivan");
// std::pair<int, string>
auto count_word = make_pair(15, string("Ivan"));
// std::pair<int, string>
auto count_word = make_pair(15, "Ivan"s);
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Классы map/multimap

Для использования необходимо подключить библиотеку <map>.

Объявление:

```
map<string, double> mp;
```

Объявление и явная инициализация map:

```
map<string, int> newMp = { { "Beta", 2 }, { "Alpha", 1 } };
```

Доступ к элементам:

- at(Key) - предоставляет доступ к указанному элементу с проверкой индекса.
- [Key] - предоставляет доступ к указанному элементу.

```
// поиск по ключу если ключ не найден, то вставляется
// элемент со значением по умолчанию (0)
double n = mp["K8"];

// поиск по ключу если ключ не найден, то вставляется
// элемент с заданным значением
mp["K8"] = 1.5;
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Классы map/multimap

Присвоение элементам map новых значений:

```
newMp.at("Alpha") = 233;
newMp["Beta"] = 7;
```

Добавление элементов:

```
newMp["Omega"] = 13;
newMp.insert(make_pair("Alpha2", 221));
```

Удаление элементов:

```
auto it = newMp.find("Alpha"); // Если элемента нет в
if (it != newMp.end())         // контейнере, то итератор
  newMp.erase(it);             // будет равен (myMap.end())

newMp.erase("Alpha2");
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Классы map/multimap

Доступ к элементам по итератору (объект pair):

- первый элемент (pair::first) – ключ
- второй элемент (pair::second) – значение
- элементы хранятся в отсортированном виде: от меньшего ключа к большему ключу

```
map<string, int>::iterator it;
for (it = newMp.begin(); it != newMp.end(); it++)
{
  cout << it->first << " " << (*it).second << endl;
}
```

Полезные методы:

count(Key) – возвращает количество элементом соответствующих определенному ключу.

find(Key) – поиск первого элемента по ключу.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования map

Заполнение словаря – телефонной книги, ключом в которой служит фамилия, а элементом – номер телефона.

```
#include <fstream>
#include <iostream>
#include <string>
#include <map>
using namespace std;
typedef map <string, int> map_si;
int main() {
  map_si ml;
  ifstream in("phonebook.txt");
  string str;
  int num;
  while (!in.eof()) {
    in >> num // Чтение номера
    in.get(); // Пропуск пробела
    getline(in, str); // Чтение фамилии
    ml[str] = num; // Занесение в словарь
    cout << str << " " << num << endl;
  }
```

Содержимое файла phonebook.txt:
1001002 Petya K.
3563398 Ivanova N.M.
1180316 Vovochka
2334476 Vasia

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования map. Продолжение

```
  ml["Petya P."] = 2134622; // Дополнение словаря

  map_si::iterator i;
  cout << "ml:" << endl; // Вывод словаря
  for (i = ml.begin(); i != ml.end(); i++)
    cout << (*i).first << " " << (*i).second << endl;

  i = ml.begin();
  i++; // Вывод второго элемента
  cout << "Второй элемент: ";
  cout << i->first << " "
       << i->second << endl;

  // Вывод элемента по ключу
  cout << "Vasia: "
       << ml["Vasia"] << endl;

  return 0;
}
```

Вывод:

Petya K. 1001002
Ivanova N.M. 3563398
Vovochka 1180316
Vasia 2334476
ml:
Ivanova N.M. 3563398
Petya K. 1001002
Petya P. 2134622
Vasia 2334476
Vovochka 1180316
Второй элемент: Petya K. 1001002
Vasia: 2334476

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс set

Объявление:

```
set<string> MySet;
```

Доступ к элементам по итератору:

```
set<string>::iterator it;
MySet.insert("K8");
MySet.insert("AS14");
MySet.insert("K8"); // не добавится т.к. элемент уже есть
for (it = MySet.begin(); it != MySet.end(); it++) {
  cout << *it << endl;
}

// Альтернативный вывод
it = MySet.begin(); // присваиваем ему начальный элемент
while (it != MySet.end()) // пока итератор не достигнет
{                         // последнего элемента
  cout << *it << endl;    // выводим значение элемента, на
                          // который указывает итератор и
  ++it;                   // переходим к следующему элементу
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева